

Analyzing the Effect of Network Width on Graph Transformers

Progress report — what was done, what came out, and what it means

Weeks 1–5 of the department calendar · 2026-09-24 → 2026-10-05 · raw log: `docs/CHANGELOG.md` · questions: `docs/WIDTH-QUESTIONS.md`

Summary.

1. **Width has to be measured with the learning rate tuned per width.** With one fixed rate, wider models look up to 0.4 worse than they are; the best rate falls as width grows.
2. **Width has a floor and a data-dependent ceiling.** Below a minimum width the model cannot fit at all; above some width it memorizes, and that ceiling moves up with more training data (2000 → 8000 graphs removes it).
3. **Our first benchmark was too easy.** It leaked component membership through node indices and graph statistics. We built an audited replacement (*swap*) on which statistics and short-range checks are at chance.
4. **On the clean task, width decides whether learning starts at all.** At $n = 64$ only $m \geq 64$ leave the trivial predictor; at $n = 128$ nothing up to $m = 128$ does.
5. **The critical width grows superlinearly in n .** Measured at depth 6 for $n = 32$ –56: the width needed to **fit** the training data grows as $n^{1.53}$, the width needed to **generalize** as $n^{2.19}$ — far above the width that theory says suffices to **represent** connectivity at this depth.

1. Question and set-up

Thesis question. How does the width of a graph transformer — its embedding dimension m — determine what graph problems it can solve and learn, and how does that need scale with graph size n ?

Task connectivity matrix. Input: a graph on n nodes. Output: the $n \times n$ matrix R with $R_{ij} = 1$ iff i and j are in the same component. Dense per-pair supervision; a single connected/disconnected bit per graph does not train (stalls at `ln 2`, June results).

Model transformer encoder whose tokens are the rows of $A + I$ (one token per node), pre-norm blocks, multi-head attention with head dimension fixed at 8 (so heads = $m/8$), pairwise bilinear read-out $HWH^\top$.

Width m , the embedding dimension. Depth L = number of blocks.

Metrics **exact-match** (the whole matrix right) and **pair accuracy** (fraction of entries right). The trivial “everything connected” predictor scores 0.5 exact-match (half the graphs are connected) and ≈ 0.75 –0.77 pair accuracy.

Protocol train / validation / test split; learning rate and checkpoint picked on **validation**, reported on test; 3 seeds unless stated. **Critical width** m^* = the smallest m at which at least 2/3 of seeds reach the threshold (0.90 / 0.95 / 0.99).

Compute sweeps run on Kaggle ($2 \times T4$) as 6–8 parallel shards (`width_sweep.py`, `kaggle/width_sweep.ipynb`); every run is one JSON line in `results/width/`.

2. Timeline

Date	Step	Outcome
09-24	Topic settled; literature sweep	35 papers, theory predictions (<code>docs/WIDTH-LITERATURE.md</code>)
09-30	Q1: fixed vs tuned LR (72 runs)	fixed LR distorts the width curve
10-01	Q1b: + warm-up/cosine (120 runs)	instability fixed; wide-model drop remains
10-01	Q1c: width $\times$ data (216 runs)	the drop is memorization; data moves the ceiling
10-01	Found the index leak in <code>hard</code>	Q1 measures a near-local task
10-02	Q3 pilot 1: <code>hard_diam</code> (18 runs)	diameter $\approx n/2$; nothing learns at $n \geq 32$
10-02	Shortcut audit; <code>swap</code> generator	statistics at chance, ≤ 4 -hop checks at 0
10-02	Q3 pilot 2: <code>swap</code> , 8000 graphs	17 of 18 runs stay trivial
10-02	Learnability probe (6 runs)	learns to 0.98–0.998 with 32 000 graphs
10-04	Q3 trimmed grid (60 runs)	clean floor at $n = 32$; onset width grows with n
10-05	Q3 fine grid ($n = 32$ –56, 84 runs)	critical width $\propto n^{1.53}$ (fit), $n^{2.19}$ (generalize)
10-05	Q3 data check (32 runs)	running: does more data close the gap?

Table 1: What was done, in order.

3. Q1 — the learning rate confounds width

Set-up. $n = 24$, depth 2, 2000 training graphs, widths 8–256, learning rates $3 \cdot 10^{-4} \dots 10^{-2}$, 3 seeds (Q1). Q1b repeats it with 5 % linear warm-up and cosine decay, adds widths 2 and 4 and rate $3 \cdot 10^{-2}$.

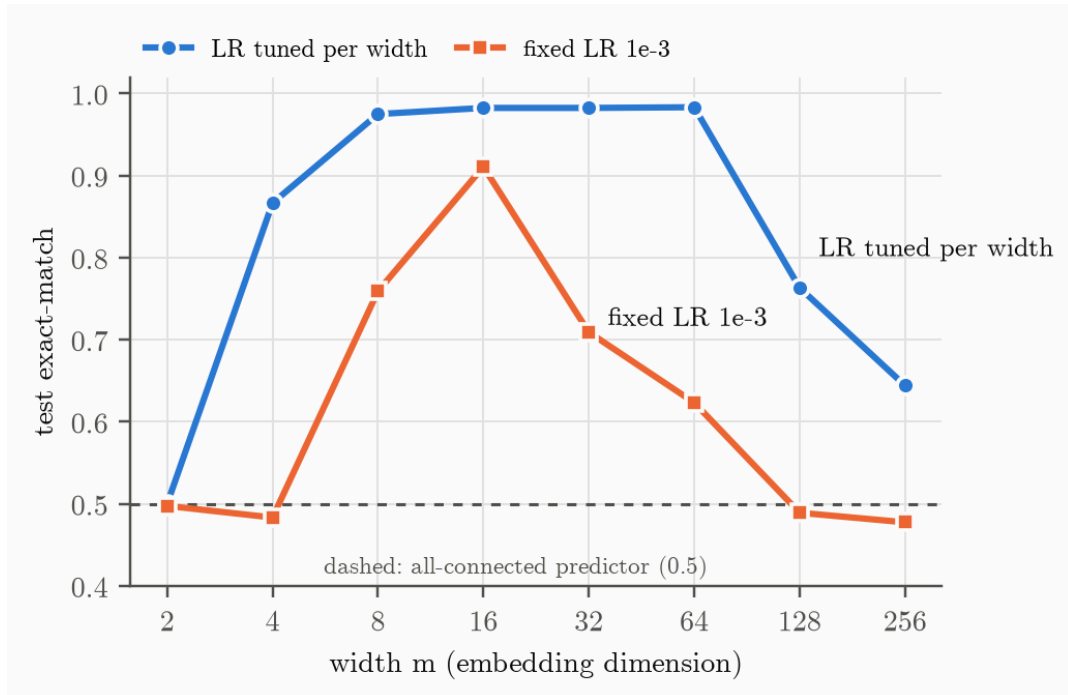


Figure 1: Q1b: test exact-match vs width with one fixed learning rate versus the rate chosen per width on validation (mean of 3 seeds).

m	2	4	8	16	32	64	128	256
Q1, fixed 1e-3	–	–	0.917	0.917	0.781	0.912	0.820	0.596
Q1, tuned	–	–	0.976	0.977	0.967	0.948	0.844	0.678
Q1b, fixed 1e-3	0.497	0.483	0.759	0.911	0.709	0.623	0.489	0.478
Q1b, tuned	0.497	0.867	0.975	0.967	0.982	0.983	0.763	0.644

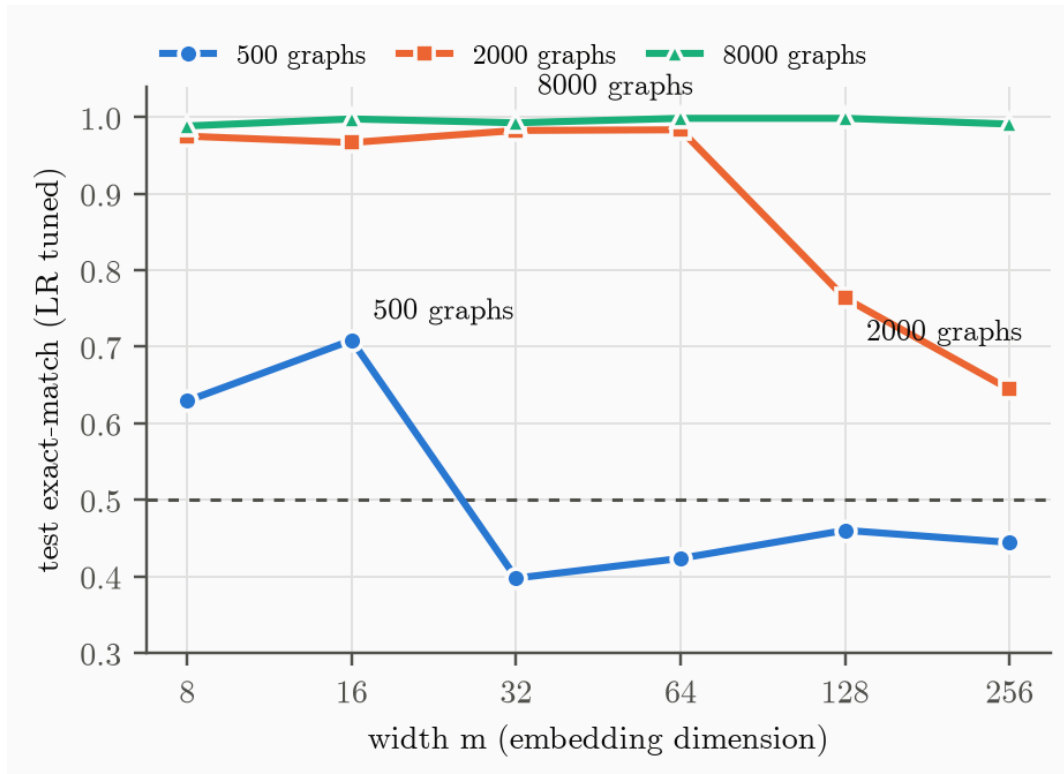
Table 2: Test exact-match, mean of 3 seeds.

✓ **Finding**

- With a fixed rate the width curve zig-zags and collapses (Q1b: 0.48 at $m \geq 128$); tuned per width, $m = 8\text{--}64$ all reach $\approx 0.97\text{--}0.98$.
- The best rate **falls as width grows** (10^{-2} at small $m \rightarrow 10^{-3}$ at $m = 256$); without warm-up, wide models diverge at high rates ($m = 256$ at 10^{-2} : train 0.41).
- Warm-up fixes the instability (wide models reach train 1.00) — but tuned test at $m = 128/256$ stays at 0.76 / 0.64.

4. Q1c — the wide-model drop is memorization

Same set-up with 500, 2000 and 8000 training graphs, at a **fixed number of optimizer steps** (4800), so more data never means more training.

Figure 2: Q1c: tuned test exact-match vs width for three training-set sizes. Training accuracy is ≈ 1.00 everywhere.✓ **Finding**

- At 8000 graphs every $m \geq 16$ reaches ≥ 0.99 — the drop at $m \geq 128$ is gone.
- At 500 graphs everything memorizes; $m \geq 32$ falls **below** the trivial predictor.
- The 2000-graph arm reproduces Q1b to the third decimal (independent Kaggle run).
- With enough data the fixed rate is within 0.01 of tuned; tuning matters most when data is scarce.

◆ What it means

Width has a **floor** (below ≈ 4 –8 the model cannot even fit the training set) and a **ceiling set by the data** (past it, extra width memorizes). The floor is what theory talks about; the ceiling is the practical limit it doesn't cover.

! Caveat

Q1 used the **hard** generator, which turned out to leak (next section). Q1's lessons are about **method** — tune the rate per width, give enough data — not about reachability.

5. Making the task honest: the shortcut audit

The **hard** generator builds two dense blobs, with or without one bridge. Three problems surfaced:

1. **Index leak.** Node labels were never shuffled: the blobs are always nodes $[0, n_a)$ and $[n_a, n)$ (200 / 200 graphs checked). With adjacency-row tokens the split point is readable, so connectivity reduces to “does any row have ones on both sides”.
2. **Statistics leak.** A bridge closes no cycle; the matched extra edge in the disconnected class always closes a short one. Gradient boosting on degree and closed-walk counts ($\text{tr } A^3 \dots \text{tr } A^6$) predicts the label with 0.68–0.90 accuracy at $n = 16$ for every existing generator.
3. **Locality.** On small graphs a model that only looks 6 hops away gets the whole matrix right on 60–100 % of graphs.

Fix — the swap generator. Two sparse blobs (cycle + $\frac{1}{4} \cdot \text{size}$ chords). Pick two nodes ≥ 6 hops apart in each blob (a_1, a_2 and b_1, b_2). Disconnected: add $a_1 a_2$ and $b_1 b_2$. Connected: add $a_1 b_1$ and $a_2 b_2$. Same degree changes in both classes; every new cycle has ≥ 7 edges; graphs failing the distance test are redrawn **before** the label is applied; node labels shuffled per graph.

n	32	40	48	56	64	128
statistics $\rightarrow$ label	0.526	0.512	0.493	0.504	0.512	0.507
statistics + 4-hop (exact)	0.256	0.257	0.246	0.256	0.259	0.253
4-hop check (exact)	0.000	0.000	0.000	0.000	0.000	0.000
diameter, connected	8.6	9.8	10.9	11.7	12.5	16.7

Table 3: Audit of swap (chord fraction 0.25, $d_{\min} = 6$) — `audit_width_data.py`. Trivial predictor: 0.5. Diameter grows $\approx \log n$.

◆ What it means

Every heuristic we could think of sits at or below the trivial predictor, so a model that scores well on **swap** has to trace paths. The price: **swap** cannot be built below $n \approx 32$, and it is much harder to learn.

6. Q3 — critical width versus graph size

Pilots and the learnability probe

Run	Set-up	Result
Pilot 1	hard_diam (diameter $\approx n/2$), $n = 16/32/64$, 8000 graphs	$n = 16$ learns (depth 4: 0.99); $n \geq 32$ all at 0.50
Pilot 2	swap, $n = 32/64/128$, 8000 graphs, 4800 steps	17 / 18 at 0.50; only $n = 32$, depth 5, $m = 128$: pair 0.87 vs 0.75 trivial
Probe	swap, $n = 32$, depth 5, 32 000 graphs, 24 000 steps	$m = 64 / 128 / 256$ reach 0.985 / 0.998 / 0.988 test exact-match

Table 4: The road to a learnable, shortcut-free setting.

✓ Finding

The clean task **is** learnable — test tracks train (0.998 vs 1.000), pair accuracy 0.998. Pilot 2 failed mainly for lack of data (with 8000 graphs the $m = 128$ model memorized: train 0.99, test 0.53), the same data ceiling as Q1c. Wider models also start later at a fixed rate (rate $3 \cdot 10^{-3}$, epoch 12: 0.83 / 0.40 / 0.07 for $m = 64 / 128 / 256$).

The trimmed grid

swap, $n = 32 / 64 / 128$ at depth $\lceil \log_2 n \rceil = 5 / 6 / 7$, $m = 8 \dots 128$, 32 000 graphs, 12 000 steps, rates 10^{-3} and $3 \cdot 10^{-3}$, 2 seeds (60 runs).

m	$n = 32$	$n = 64$	$n = 128$
8	0.50 (train 0.27)	0.50	0.50
16	0.49 (train 0.39)	0.50	0.50
32	0.90	0.50	0.50
64	0.97	0.50	0.50
128	0.97	0.50	0.50

Table 5: Tuned test exact-match (mean of 2 seeds).

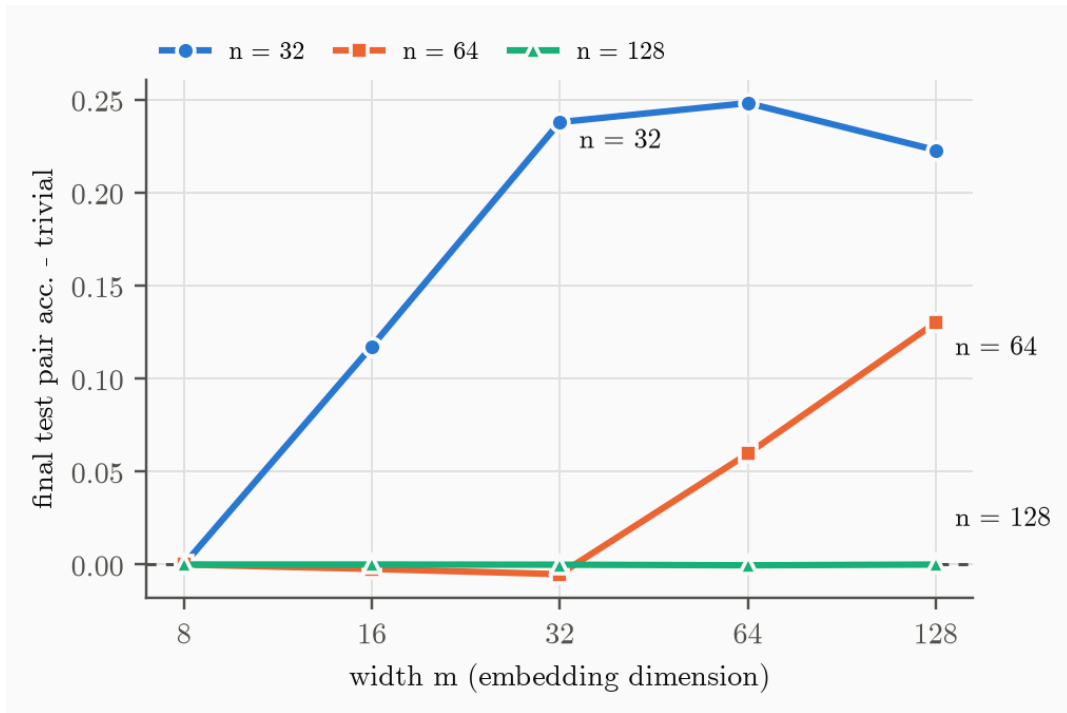


Figure 3: How far each model gets above the trivial predictor (final test pair accuracy at rate $3 \cdot 10^{-3}$, mean of 2 seeds, no selection on test). The width at which learning starts moves right as n grows.

✓ Finding

- $n = 32$: a sharp floor. $m = 8$ cannot fit the training set; $m = 16$ starts but stalls; $m = 32$ reaches 0.90; $m \geq 64$ reach 0.97. **m^* is between 32 and 64** (by pair accuracy: 32 at ≥ 0.95 , 64 at ≥ 0.99).
- $n = 64$: $m \leq 32$ never leave the trivial predictor; $m = 64$ starts (train loss $0.59 \rightarrow 0.07$ – 0.10 , pair 0.80 – 0.85); $m = 128$ gets further (pair 0.89 – 0.90). Exact-match stays 0.5 — about 4000 pairs must all be right.
- $n = 128$: nothing moves for any $m \leq 128$.

◆ What it means

The width at which learning **begins** is ≈ 16 at $n = 32$, ≈ 64 at $n = 64$ and above 128 at $n = 128$ — growing at least linearly in n . This is the direction of the theory: a single step that gathers a node’s whole reach set is q -sparse averaging with q close to n , which needs width of order n (Sanford, Hsu, Telgarsky 2023; see [reports/qs-a-sparse-averaging.pdf](#)). On the old, leaky task the same model needed only $m = 8$ at $n = 24$ — the shortcuts were hiding the width requirement.

The fine grid: a scaling law

swap, $n = 32 / 40 / 48 / 56$ at a fixed depth 6 ($= \lceil \log_2 n \rceil$ for all four), $m = 16 \dots 128$, 32 000 graphs, 24 000 steps, rate $3 \cdot 10^{-3}$, 3 seeds (84 runs).

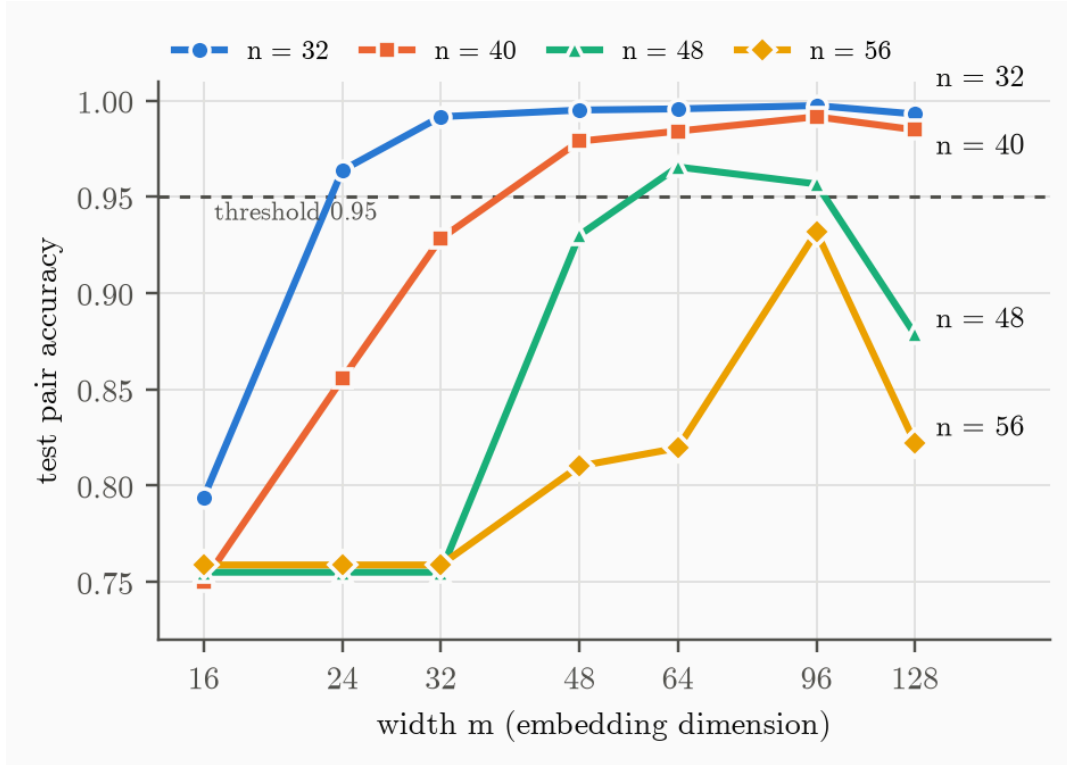


Figure 4: Test pair accuracy vs width (mean of 3 seeds; trivial ≈ 0.75). Each curve shifts right as n grows. The drop at $m = 128$ is the learning-rate effect from Q1: $3 \cdot 10^{-3}$ is too high there, and training accuracy falls too.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
width to fit (train pair ≥ 0.95)	18.9	23.3	31.4	44.9	$\propto n^{1.53}$
width to generalize (test pair ≥ 0.95)	23.2	38.1	56.4	> 96	$\propto n^{2.19}$

Table 6: Critical width: where the seed-mean curve crosses 0.95, interpolated in $\log_2 m$; slope of the log-log fit across n ($m \leq 96$; `width_sweep.py q3fine --scaling`). Per-seed crossings agree within $\approx \pm 3$.

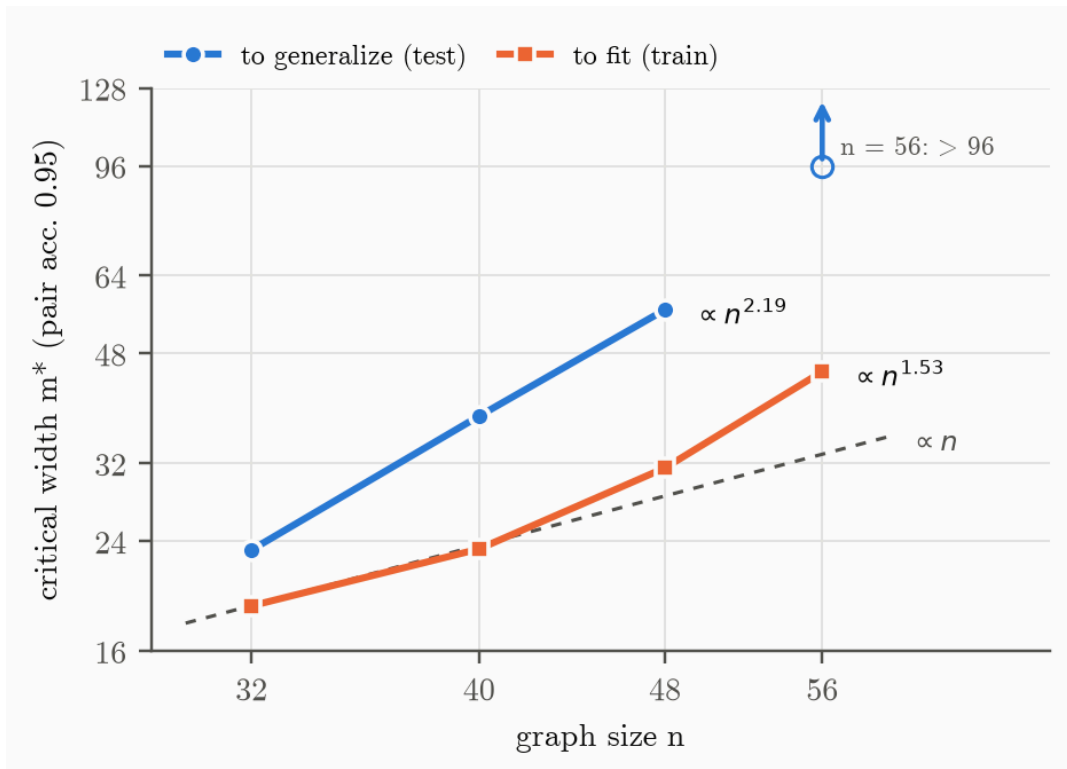


Figure 5: Critical width vs graph size, log-log. Both lines are steeper than $m \propto n$ (dashed).

✓ **Finding**

- Both critical widths grow **faster than linearly** in n : $\approx n^{1.5}$ to fit, $\approx n^{2.2}$ to generalize.
- The gap between them widens with n : at $n = 56$, $m = 64$ fits the training set (pair 0.984) but reaches only 0.820 on test.
- Seeds agree closely; the scaling is not noise.

◆ **What it means**

Theory says depth $\log n$ needs only width n^ϵ to **represent** connectivity (Sanford et al. 2024a). What we measure is the width to **learn** it, and that grows faster than n . The widening fit/generalize gap is the Q1c data ceiling again: 32 000 graphs give fewer examples per pair as n grows, so the 2.19 is probably inflated by holding data fixed. The data check ($2\times$ and $4\times$ the graphs at $n = 48 / 56$) tests this; 1.53 is the cleaner number until then.

7. What it all means so far

1. **Measuring width is a protocol problem first.** Learning rate and training-set size both interact with width strongly enough to fake or hide a width effect. Every later sweep tunes the rate per width and checks the data ceiling.
2. **Benchmarks decide the answer.** The same architecture needs width 8 on the leaky task and ≥ 32 –64 on the audited one at similar n . A width study is only as good as its shortcut audit — a methodological contribution in its own right.
3. **Representation is not learning.** Theory says width $\approx n$ (or depth $\log n$) is **enough to represent** connectivity. In practice the width needed to **learn** it grows faster than n ($n^{1.5}$ to fit, $n^{2.2}$ to generalize at our budget), and data and steps have to grow too; at $n = 128$ nothing up to $m = 128$ learns. This matches Saparov et al. (ICLR 2025): transformers struggle to learn search even when it is representable.

! **Caveat**

- The slopes come from four graph sizes over less than a factor of two in n , at one training budget; they are measured, but not yet shown to be budget-free.
- Exact-match gets stricter as n^2 grows; pair-accuracy thresholds are reported alongside from now on.
- The **swap** diameter grows with $\log n$, so n and path length still rise together; Q7 will separate them.
- One learning rate in the fine grid ($3 \cdot 10^{-3}$), for budget reasons; it is too high at $m = 128$, which is left out of the fit.

8. Next steps

Item	Purpose	Status
Q3 data check	$n = 48 / 56$, $m = 32 \dots 96$, 64 000 and 128 000 graphs: is the generalize slope a data effect?	running
Q6 input bottleneck	edge tokens instead of n -wide adjacency rows: is $m^* \approx n$ just the read-in?	next
Q7 depth $\times$ width	fix n , vary diameter and depth: can width replace depth?	planned
Q4 Graphormer bias	shortest-path bias gives connectivity away; needs a task it doesn't leak	planned
Q2 task hierarchy	retrieval $<$ connectivity $<$ shortest path, each with its own audit	planned

Table 7: Week 6 is the first progress review (Явн 1); research freeze in week 11.

Reproduce. Every number above comes from `python width_sweep.py <sweep> --analyze` or `--curves` over `results/width/<sweep>*.jsonl` (sweeps `q1`, `q1b`, `q1c`, `q3pilot`, `q3pilot2`, `q3probe`, `q3trim`,

`q3fine`) and `--scaling` for critical widths; the audit from `python audit_width_data.py` figures from `reports/figures/width_progress_plots.py`.